

**Факультет Программной инженерии и компьютерной техники
Образовательная программа Проектирование и разработка систем больших данных
Направление подготовки (специальность) 09.04.04 - Программная инженерия**

ОТЧЕТ

по лабораторной работе №1
по курсу «Структуры и алгоритмы в базах данных и распределенных системах»
на тему: «Реализация алгоритмов хэширования»

Обучающийся: Никифорова Е. А., Р4135

Преподаватель: Платонов А. В., доцент

Преподаватель: Портнов П. В., ассистент

Содержание

Задание	3
Реализация	4
Расширяемое хэширование (Extendible Hashing)	4
Структура данных в памяти и на диске	4
Основные структуры	4
Интерфейс	4
Механика работы	4
Идеальное хэширование (Perfect Hashing)	6
Структура данных в памяти	6
Основные структуры	6
Интерфейс	6
Тестирование	7
Расширяемое хэширование (Extendible Hashing)	7
Результаты бенчмарков:	7
Результаты профайлинга perf:	8
Идеальное хэширование (Perfect Hashing)	8
Результаты бенчмарков:	8

Задание

Необходимо реализовать три алгоритма (в скобках операции):

1. Extendible хэш таблицу на файловой системе с бакетами(вставка, обновление и удаление). Прямая работа с диском и mmap
2. Perfect hash (создание полного индекса по фиксированному набору ключей и поиск)
3. LSH для поиска дублей в текстах или точек в 3d (создание индекса, добавление новых точек, фулл скан для поиска дублей)

По всем алгоритмам должны быть:

1. Тесты функциональности на случайных наборах данных (набор данных при тесте генерируется)
2. Перфтесты на больших объемах данных - считаем пропускную способность и время операции
3. Профайлинг памяти и цпу всех реализаций с анализом узких мест и набором гипотез как можно сделать лучше

Реализация

Расширяемое хэширование (Extendible Hashing)

Структура данных в памяти и на диске

Файл БД организован как непрерывный регион, разделенный на три секции:

1. **Header**: Хранит глобальные параметры (глубины, смещения, размеры).
2. **Directory**: Массив 64-битных смещений. Каждый элемент — это `bucket_offset` (смещение от начала файла до конкретного бакета).
3. **Buckets**: Блоки фиксированного размера. Каждый бакет содержит `BucketHeader` (локальная глубина и счетчик) и массив `Entry`.
4. **Entry**: Единица хранения - пара ключ-значение

Основные структуры

```
struct Header {
    uint32_t global_depth;           // Глобальная глубина
    uint32_t max_global_depth;      // Ограничение максимальной глубины
    uint64_t bucket_capacity;       // Сколько структур Entry влезает в один
    бакет.                          // Задается при создании таблицы.

    uint64_t bucket_count;          // Общее кол-во физических бакетов в
    файле.

    uint64_t first_bucket_offset;    // Смещение до начала бакетов
    uint64_t file_size;             // Общий размер файла
};

struct BucketHeader {
    uint32_t local_depth; // Локальная глубина
                        // Показывает, сколько бит хеша общие
                        // для всех ключей в этом бакете

    uint32_t count;
};

struct Entry {
    int64_t key;
    int64_t value;
};
```

Интерфейс

- `openTable / createTable`: Открытие существующего файла или создание нового с инициализацией метаданных.
- `insertRecord`: Добавление новой пары (K, V) . Если ключ существует — ошибка, если бакет полон — инициирует процедуру расщепления (Split).
- `getRecord`: Поиск значения по ключу
- `removeRecord / updateRecord`: Удаление или обновление значения существующего ключа.

Механика работы

- Используется системный вызов `mmap` для отображения файла базы данных напрямую в виртуальное адресное пространство процесса.
- Для любого ключа вычисляется индекс в директории через битовую маску:
 - $\text{index} = \text{hash}(\text{key}) \ll (\text{global_depth} - 1)$

- Из директории извлекается смещение, которое складывается с базовым адресом для получения физического адреса бакета.
- Переполнение (Split)
 - Если `local_depth < global_depth`:
 - создается новый бакет, элементы перераспределяются, часть указателей директории перекидывается на новый адрес.
 - Если `local_depth == global_depth`:
 - сначала удваивается директория, `global_depth++`, затем стандартный Split.
- Расширение (Remapping):
 - При создании бакета файл физически растет через `ftruncate`.
 - Старый mmap сбрасывается, и создается новый на увеличенный размер.

Идеальное хэширование (Perfect Hashing)

Структура данных в памяти

- Perfect Hashing (FKS) строится один раз для статического набора ключей, обеспечивая поиск за $O(1)$ в худшем случае.
- Первый уровень (Глобальный): Хранит параметры хеш-функции и массив указателей на подтаблицы.
- Второй уровень (Локальный): Набор независимых подтаблиц (SubTable).
 - Если в бакет попало n_i элементов, размер подтаблицы устанавливается как n_i^2 .

Основные структуры

```
++
struct Entry {
    int64_t key;
    int64_t value;
    static constexpr int64_t kEmpty = std::numeric_limits<int64_t>::min();
};

struct HashParams {
    uint64_t a; // Случайный множитель
    uint64_t b; // Случайный сдвиг
    uint64_t m; // Размер таблицы
};

struct SubTable {
    HashParams params;           // Локальные параметры для 2-го уровня
    std::vector<Entry> cells;    // Массив пар Key-Value
    bool initialized;           // Флаг наличия данных в корзине
};

struct PerfectHash {
    size_t _n;                  // Общее кол-во уникальных
    // ключей
    HashParams _first_level_params; // Параметры основной функции
    std::vector<SubTable> _second_level_tables;
};
```

Интерфейс

- PerfectHash(keys): Конструктор. Выполняет предварительную фильтрацию дубликатов, распределение по корзинам и итеративный подбор параметров a, b для каждого уровня до тех пор, пока не будет достигнуто отсутствие коллизий.
- get(key): Поиск. Возвращает std::optional. Выполняет ровно два вычисления хеша и одну проверку ключа в ячейке.

Тестирование

Расширяемое хэширование (Extendible Hashing)

Результаты бенчмарков:

=== РЕЗУЛЬТАТЫ ЗАДЕРЖКИ (среднее, ns/op) ===								
N		INSERT		UPDATE		DELETE		GET

1000		452.73		21.93		18.75		0.03
5000		666.61		23.01		18.55		0.00
10000		791.45		26.80		19.21		0.00
50000		1242.61		30.60		22.84		0.00
100000		1721.47		35.76		24.01		0.00
500000		2604.26		73.03		50.31		0.00
1000000		3402.09		144.41		110.50		0.00
=== РЕЗУЛЬТАТЫ ПРОПУСКНОЙ СПОСОБНОСТИ (среднее, млн оп/с) ===								
N		INSERT		UPDATE		DELETE		GET

1000		2.29		45.77		53.50		37916.67
5000		1.50		43.54		53.95		208168.64
10000		1.26		37.33		52.09		442687.75
50000		0.81		32.83		43.83		1955874.74
100000		0.59		28.47		42.03		3718473.76
500000		0.38		13.72		19.89		10812009.28
1000000		0.29		6.99		9.33		8312170.41

- INSERT: Самая тяжелая операция. Задержки растут из-за физического расширения файла (ftruncate), перераспределения страниц памяти и процедур расщепления бакетов.
- GET: Благодаря mmap доступ к данным происходит практически со скоростью оперативной памяти, пока файл умещается в кэш.
- UPDATE и DELETE показывают предсказуемое время выполнения на малых объемах, так как не требуют изменения структуры таблицы, в отличие от вставки
- При $N > 500,000$ наблюдается резкое пропускной способности всех операций (кроме GET). Это связано с ростом файла и увеличением количества промахов кэша процессора при обходе директории.

[illegible]

- ## Идеальное хэширование (Perfect Hashing)

```

=== РЕЗУЛЬТАТЫ ЗАДЕРЖКИ (среднее, ns/op) ===
N          | BUILD          | GET
-----
1000       |                | 156.03 | 14.22
5000       |                | 235.64 | 25.94
10000      |                | 221.88 | 56.28
50000      |                | 230.95 | 84.90
100000     |                | 315.06 | 113.90
500000     |                | 488.66 | 133.60
1000000    |                | 455.54 | 108.87

=== РЕЗУЛЬТАТЫ ПРОПУСКНОЙ СПОСОБНОСТИ (среднее, млн оп/с) ===
N          | BUILD          | GET
-----
1000       |                | 6.41 | 70.31
5000       |                | 4.24 | 38.55
10000      |                | 4.51 | 17.77
50000      |                | 4.33 | 11.78
100000     |                | 3.17 | 8.78
500000     |                | 2.05 | 7.49
1000000    |                | 2.20 | 9.19

```


С увеличением числа элементов построение таблицы (BUILD) становится значительно медленнее и менее производительным. При этом операции получения (GET), хоть и показывают некоторое замедление, остаются весьма эффективными. Основная вычислительная нагрузка при использовании идеального хеширования приходится на этап построения самой таблицы.